Name: _____ ID: _____

CSE 2046 / CSE 2246 Final Exam

20.06.2025 – 105 minutes

Q1	Q2	Q3	Q4	Q5	Q6	Q7	SUM
/20	/15	/15	/15	/20	/15	/20	/100

Q-1 (20 pts) *Dynamic Programming.* **Maximum Value Path Ending at Zero:** You are given a 2D grid of size N × M (i.e. a matrix grid[N][M]). Each cell of the grid:

- Contains a **non-negative integer** value (from 0 to 100), or
- May be **blocked**, represented by the value **−1** (impassable).

A robot starts from the **top-left corner** cell (0,0) and may only move **right** or **down** at each step. Its goal is to **reach any cell that has a value of 0**, while collecting the **maximum possible sum of cell values** along the way. You are going to design a dinaymic programming (DP) algorithm for this problem. Your algorithm will find the max total value the robot can collect by reaching one of the zero-valued cells.

(a – 10 pts) Set up a recurrence relation and briefly describe the DP steps.

V[i,j] : max value from (0,0) to (i,j).

V[0,0] = grid[0,0] V[i,j] = −∞ if grid[i,j] = −1.

V[0,j] = V[0,j−1] + grid[0,j] ∀ j ∈ [1, m−1] // first row

V[i,0] = V[i−1,0] + grid[i,0] ∀ i ∈ [1, N−1] // first column

(you may also fill the out of table with −∞)

```
max ← −∞
for i = 1 to N−1
    for j = 1 to m−1
        if grid[i,j] ≠ −1 {
            V[i,j] = max{ V[i−1,j], V[i,j−1] } + grid[i,j];
            if grid[i,j] = 0 and V[i,j] > max then max ← V[i,j];   // choose max among all zeros.
        }
```

(b – 7 pts) Apply your algorithm to the following instance (note that there are two cells with value 0 which are the potential end points):

$$\begin{bmatrix} 1 & 2 & 3 & -1 & 8 \\ 4 & 3 & 2 & 6 & \mathbf{0} \\ -1 & 2 & 1 & -1 & 7 \\ 6 & 3 & 1 & \mathbf{0} & 4 \end{bmatrix}$$

After filling first row, first column: $V = \begin{bmatrix} 1 & 3 & 6 & -\infty & -\infty \\ 5 & & & & \\ -\infty & & & -\infty & \\ -\infty & & & & \end{bmatrix}$

Then we fill the second row, then the third row, and so on.

V[1,1] = max{5,3} + 3 = 8 V[1,2] = max{8,6} + 2 = 10

V[1,3] = max{10, −∞} + 6 = 16 V[1,4] = max{16, −∞} + 0 = 16; max = 16 (16 > −∞)

V[2,1] = max{−∞, 8} + 2 = 10 V[2,2] = max{10, 10} + 1 = 11

V[2,4] = max{−∞, 16} + 7 = 23 V[3,1] = max{−∞, 10} + 3 = 13

V[3,2] = max{13, 11} + 1 = 14 V[3,3] = max{14, −∞} + 0 = 14; 14 < 16, max = 16

V[3,4] = max{14, 23} + 4 = 27

max = 16

$$V = \begin{bmatrix} 1 & 3 & 6 & -\infty & -\infty \\ 5 & 8 & 10 & 16 & 16 \\ -\infty & 10 & 11 & -\infty & 23 \\ -\infty & 13 & 14 & 14 & 27 \end{bmatrix}$$

(c – 3 pts) Describe the time complexity of your algorithm.

For each entry of the matrix V we are doing constant number of operations. So the time complexity is Θ(m.n) (# repetitions of the operations in the inner for loop is Θ(m.n). The other parts are linear.)

Name: _____ ID: _____

Q-2. (15 pts) Consider the following text consisting of 3,000 characters, formed by repeating the string "123" one thousand times: 123123123123...123.
Now, you are going to search 12323 in this text by using Boyer Moore algorithm.

(a – 5 pts) Show the bad symbol table (2 pts) and the good suffix table (3 pts).
(b – 5 pts) Apply Boyer-Moore algortihm, by showing all the steps.
(c – 5 pts) Calculate total number of character comparisons performed.

(a) Bad symbol table:

char	shift
1	4
2	1
3	2
others	5

Good suffix table

		d_2
k=1	123 23	5
k=2	12 3 23	2
k=3	12 3 23	5
k=4	12 3 23	5

(predecessor of the other 3 rs also 2)

(b) 123123123123 123123

k=2 $d_2 = 2$ } same, shift by 2.
$d_1 = 4-2 = 2$ }

(c) After the first 3 chars, we always do 4 comparisons in 3 chars.

$\frac{4}{3} \cdot 2997 = 3996$ //

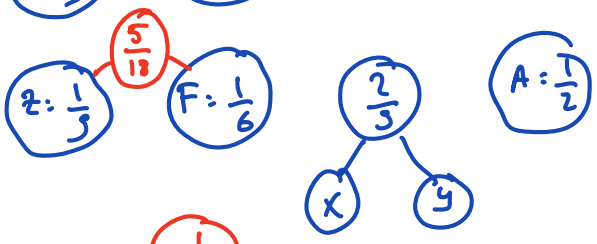
Q-3. (15 pts) Ayşe analyzes a text and she observes that the text consists of 5 characters: A, F, X, Y and Z. She observes that the half of the characters are A, and one sixth (1/6) of the characters are F. The rest of the characters (X, Y and Z) have the same frequency. She wants to apply Huffman coding to compress the text. Help her and find the codewords for each character. Also describe the compression rate compared to the fixed length encoding.

A: $\frac{1}{2}$ F: $\frac{1}{6}$ Rest: $1-\left(\frac{1}{2}+\frac{1}{6}\right)=\frac{1}{3}$ X: $\frac{1}{3}$ Y: $\frac{1}{3}$ Z: $\frac{1}{3}$

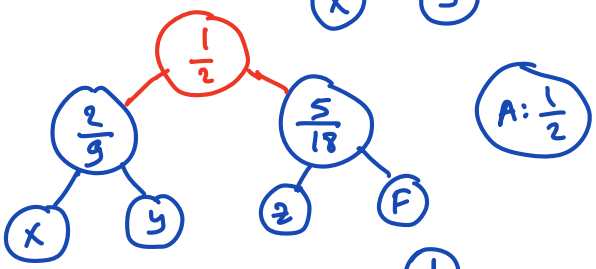
Sort:



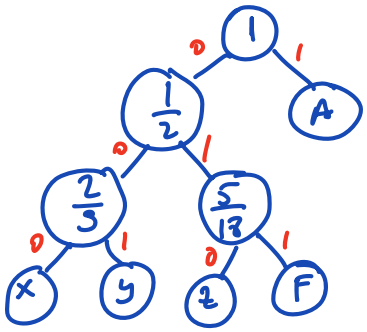
Resort:



Resort:



Final Huffman Tree:



Codewords:
A: 1
F: 011
X: 000
Y: 001
Z: 010

For min-length fixed encoding, we would need 3 bits to encode 5 chars, since $2^2 < 5 \leq 2^3$.

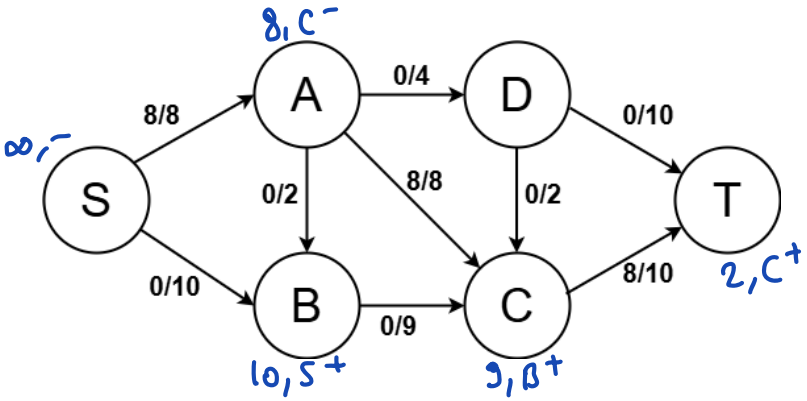
Avg codeword legth for Huffman coding =
$1 \cdot \frac{1}{2} + 3 \cdot \frac{1}{2} = 2$ bits.

Compression rate:
$\frac{3-2}{3} \times 100 = 33.\overline{3}\ \%$

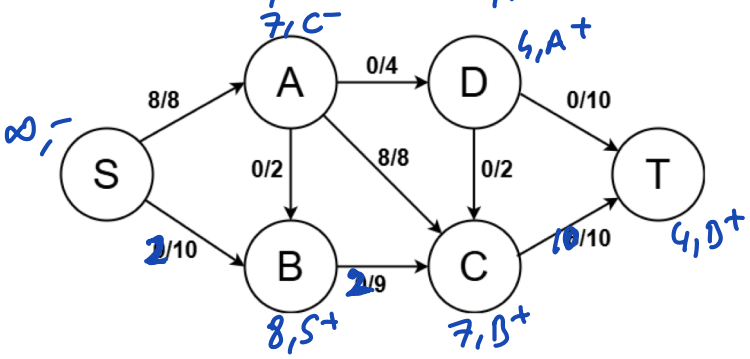
Name: _____ ID: _____

Q-4 (15 pts). *Maximum flow.* Consider the following network where S is the source and T is the sink. Some of the flows are already assigned as shown in the figure.

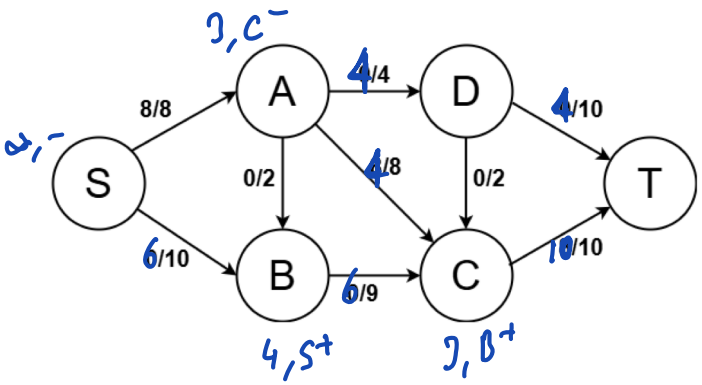
(a – 12 pts) Continue from the given step and apply the shortest augmenting path algorithm to find the maximum flow. Show the BFS queue and all the labels in every step.



Q: S B C A T
↓ ↓
aug path: S → B → C → T
flow: 2



Q: S B C A D T
↓ ↓ ↓ ↓
aug. path = S → B → C ← A → D → T
flow = 4



Q: S B C A
no path.
max. flow: 8 + 6 = 14
(or 4 + 10 = 14)
flows leaving the source
flows entering the sink

(b – 3 pts) What is the minimum cut found by the algorithm?

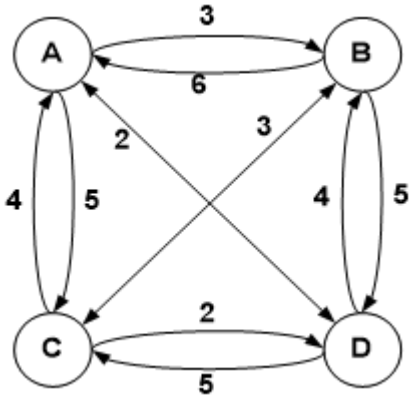
Edges from labeled nodes to unlabeled nodes at the end of the final step.
Hence, min. cut = {(A, D), (C, T)} with a value of 14.

Name: _____ ID: _____

Q-5. (20 pts) Consider a weighted directed graph G, where weigths of edges are the utilities. Now, consider a variation of Traveling Salesman Problem called MAX-D-TSP, where the goal is to find a tour with maximum utility (not minimum cost as we did in the class. Also notice that the graph is not undirected as we did in the class). Apply the branch-and-bound algorithm to find the tour with maximum utility in the following graph:

Note: Cost between A and D (and also between B and C) is same for both directions.

Another note: A tour in opposite direction may not have the same cost.



IMPORTANT! First define a bounding function. Please clearly describe how you calculate the lower/upper bound:

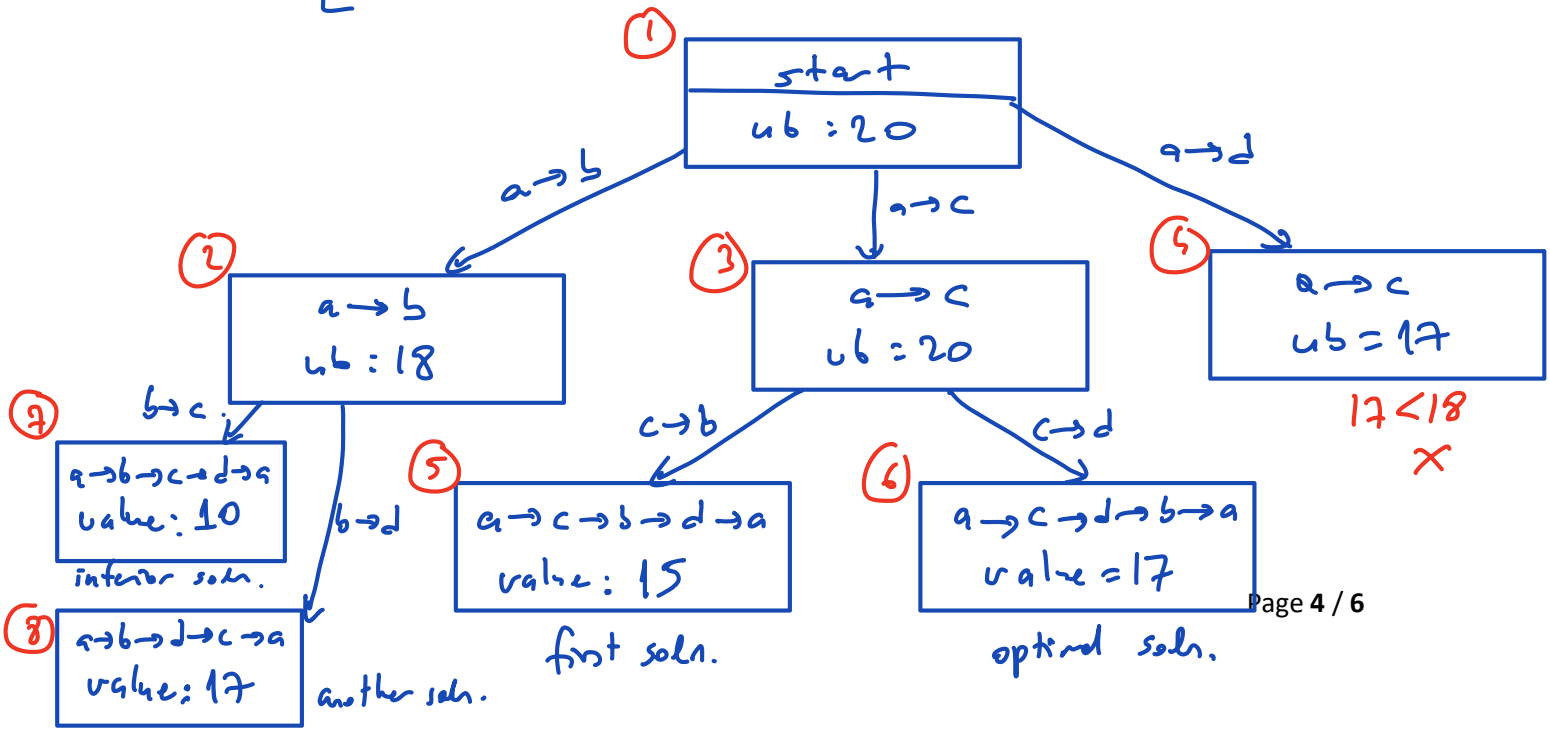
upper bound function: for each vertex, choose max outgoing edge and max incoming edge (if not already chosen); sum them all and divide by 2. $\lfloor \frac{sum}{2} \rfloor$

start: $\left\lfloor \frac{5+6+6+4+4+5+5+5}{2} \right\rfloor = 20$

$a \to b$: $\left\lfloor \frac{3+6+6+3+4+5+5+5}{2} \right\rfloor = \left\lfloor \frac{37}{2} \right\rfloor = 18$

$a \to c$: $\left\lfloor \frac{5+6+6+4+4+5+5+5}{2} \right\rfloor = 20$

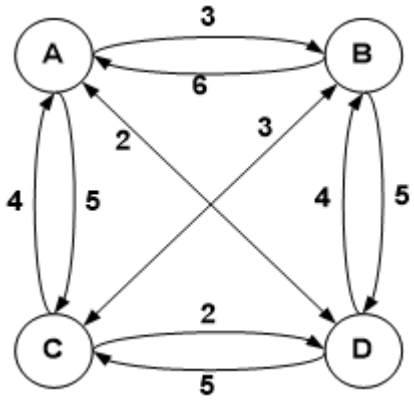
$a \to d$: $\left\lfloor \frac{2+6+6+4+4+5+2+5}{2} \right\rfloor = 17$



Name: _____ ID: _____

Q-6. (15 pts) (a – 9 pts) Describe a greedy algorithm (in a very short sentence) for the MAX-D-TSP problem given in Q-5 which is analogous to nearest-neighbor algorithm for ordinary TSP, and apply the greedy algorithm on the same instance. What is the accuracy ratio?

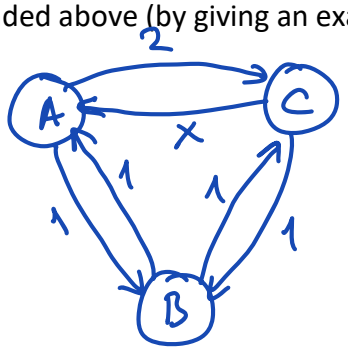
Starting from any vertex (say A), chose the max outgoing edge if the neighboring vertex is unvisited. After visiting all the vertices, go back to the starting vertex.



start from A.
Choose C : A → C
choose B : A → C → B
Choose D : A → C → B → D → A.
value: 5+3+5+2 = 15

Accuracy ratio: $\frac{17}{15}$

(b – 6 pts) Show that the performance ratio of the greedy algorithm for MAX-D-TSP is unbounded above (by giving an example).



algorithm will chose A → C → B → A tour with a total value of 4.

But if X is arbitrarily large, optimal solution will be X+2, and when X → ∞ accuracy ratio $\frac{X+2}{4}$ goes to infinity, so the performance ratio is unbounded above.

Q-7. (20 pts) (a – 6 pts) What is the trivial lower bound of the problem given in Q-1 (**Maximum Value Path Ending at Zero)**? Is this lower bound tight? Describe.

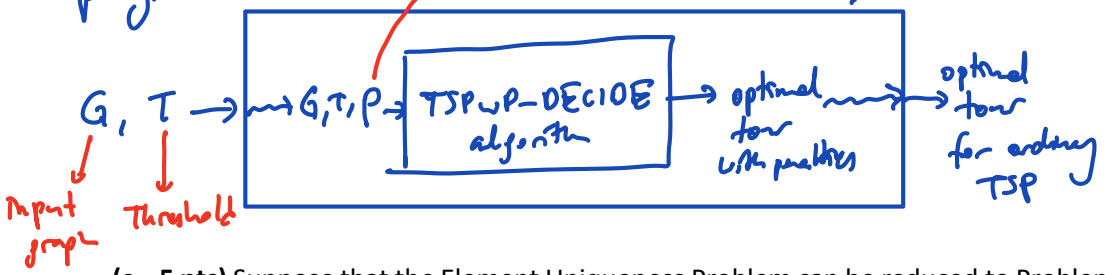
Trivial lower bound is $\Omega(nm)$ since there are nm elements and each element should be processed at least once. $\Omega(nm)$ is a tight LB, because we have an algorithm with $\Theta(nm)$ time complexity (DP algorithm).

Name: _____ ID: _____

(b – 9 pts) Consider the decision version of the TSP with Penalties (as given in your second assignment), where the objective is to determine whether there exists a tour such that the total cost of the tour, combined with the penalties incurred for unvisited nodes, does not exceed a given threshold T. Prove that this decision problem (namely TSPwP-DECIDE) is in NP, and NP-Complete, given that the decision version of ordinary TSP is NP-Complete.

TSPwP-DECIDE is in NP, because given a potential solution, we can verify it in linear time (summing the edge costs and penalties, comparing with T).

We can prove that TSPwP-DECIDE is NP-Complete if we can reduce decision version of ordinary TSP to TSPwP-DECIDE in polynomial time.



If we set penalties to infinity, output of TSPwP-DECIDE would be equal to the output of ordinary TSP.

(c – 5 pts) Suppose that the Element Uniqueness Problem can be reduced to Problem X. Problem X can be reduced both to the sorting problem and to Problem Y. All the reductions can be done in linear time. Which of the following are definitely true? Describe. **True answers without any description will not get any point.**

E.U → X → Sorting
X → Y

i. Problem X is tractable.

Problem X is tractable because we can reduce it to the sorting problem which is tractable. So we can solve it in $O(n) + O(n \log n)$ time. ($O(n)$: reduction; $O(n \log n)$: running sorting algorithm)

ii. Problem Y is tractable.

From the above info, we know that Problem Y is at least as hard as Problem X which is tractable. But we cannot know how hard it is. So we cannot say that "Problem Y is tractable" is definitely true.